

Resilience Test Harness Architecture

SakthiDB (PostgreSQL) and DistDB · Non-containerized · E1–E4

Document	Resilience Test Harness — Implementation Architecture
Version	1.0
Date	28 July 2026
Owner	Database Reliability Engineering
Implements	Unified Resilience Testing Framework v1.0 — 155 scenarios · 218 environment-qualified runs · 3 tiers · 2 target systems
Deployment model	Non-containerized hosts and VMs; environment classes E1–E4
Classification	CONFIDENTIAL — INTERNAL USE ONLY
Review cycle	Per phase gate, or on any change to the framework catalog

- 1. [Constraints That Drive This Design](#)
- 2. [Architecture Overview](#)
 - 2.1 [Execution flow](#)
- 3. [L0 — Environment and Provisioning](#)
 - 3.1 [Environment profile](#)
- 4. [L1 — Control Plane and the Catalog](#)
 - 4.1 [The catalog is the source of truth](#)
 - 4.2 [Orchestrator](#)
 - 4.3 [State-triggered injection \(compound scenarios\)](#)
- 5. [L2 — Fault Injection](#)
- 6. [L2 — Workload and Transaction Markers](#)
 - 6.1 [Two workload engines](#)
 - 6.2 [Transaction marker protocol — the RPO measurement of record](#)
 - 6.3 [Operation history export — feeding Elle](#)

DistDB key selection

- 7. Measurement Architecture
 - 7.1 Prometheus is context, not measurement
 - 7.2 RTO decomposition — signal sources
 - 7.3 Probe design
- 8. L3 — Target Adapters and Role Resolution
- 9. L4 — Observability
- 10. L5 — Validation, Analysis and Evidence
 - 10.1 Integrity tooling — the correct tool per corruption class
 - 10.2 Threshold evaluator
 - 10.3 Consistency checking with Elle
 - 10.4 Outputs
- 11. Environment Qualification Suite Packaging
 - 11.1 Packaging recommendation
- 12. Technology Stack — Decisions and Rejected Alternatives
- 13. Repository Structure:
- 14. Execution Model, Cadence and Capacity
 - 14.1 Capacity arithmetic
- 15. Safety Architecture
- 16. Build Sequence
- 17. Settled Technical Decisions
 - 17.1 Jepsen: adopt the checker, not the framework
- 18. Open Technical Decisions
- 19. Document Approval

1. Constraints That Drive This Design

This is not a greenfield design. Six constraints are fixed by the framework, and each one eliminates otherwise-obvious choices. They are listed first because most of the decisions later in this document are consequences of them rather than independent judgements.

Constraint	Consequence for the harness	What it rules out
Not containerized	Injection is via SSH to hosts, systemd, kernel facilities and hypervisor/BMC APIs	Chaos Mesh CRDs, LitmusChaos, Kubernetes operators, pod-level anything
Product on customer infrastructure (E1–E4)	Environment is a runtime parameter; a subset must ship as a customer-runnable binary	A harness hard-wired to one lab; anything assuming we own the hypervisor

Two target systems, one catalog	Target resolution by <i>role</i> ; the same test body runs against standalone, PC, QC and workers	Separate PostgreSQL and DistDB test suites
Criteria are numeric thresholds	Every scenario must emit measured values, not log lines. Pass/fail is computed, not read.	Eyeball-the-output testing; screenshot evidence
RTO must be decomposed	Sub-second, single-clock event capture across the database, the HA agent and the consensus store	Relying on Prometheus scrape intervals (§7.1)
NL-D destroys the instance	Fast, repeatable rebuild is a first-class harness capability, not a manual step	Hand-built lab nodes; any workflow where reset takes tens of minutes

2. Architecture Overview

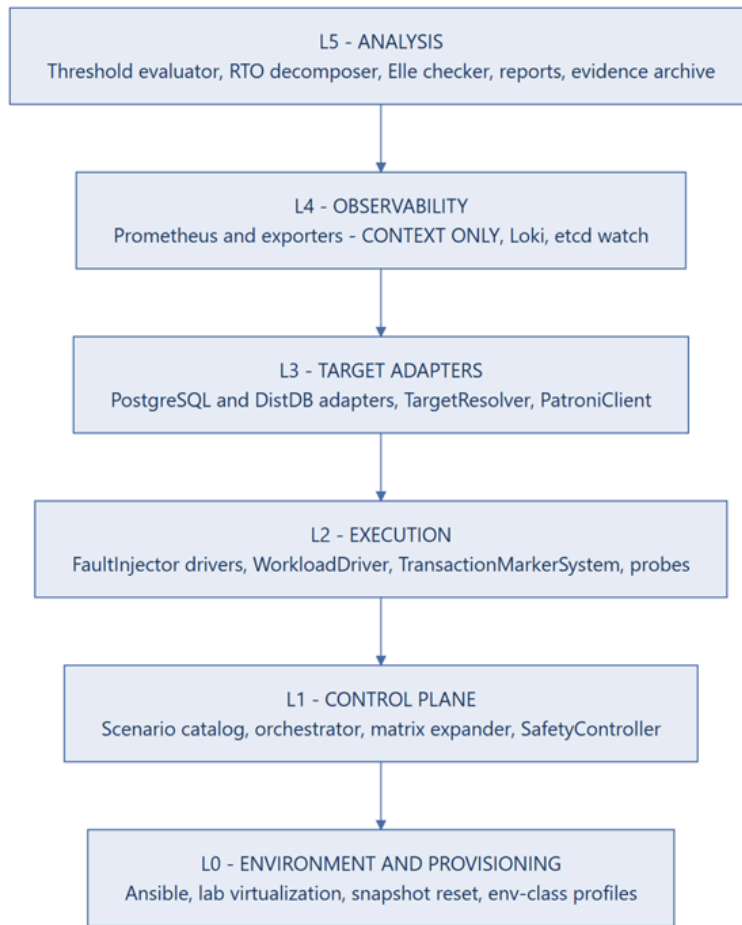


Figure 1 — Six layers. Each depends only on the layer below it.

Layer	Owns	Key components
L0 Environment	Provisioning any supported environment class, and resetting it to a known baseline between runs	Ansible, Terraform, libvirt/KVM, LVM or ZFS snapshot reset, env-class profiles
L1 Control plane	What runs, in what order, and whether it is safe to run	Scenario catalog, matrix expander, 8-phase orchestrator, SafetyController
L2 Execution	Applying the fault and driving the workload	FaultInjector drivers, WorkloadDriver,

		TransactionMarkerSystem, probes	
L3 Target adapters	Speaking to whichever database role is under test	PostgreSQLAdapter, DistDBAdapter, TargetResolver, PatroniClient	
L4 Observability	Context, corroboration and trend — never pass/fail	Prometheus and exporters, Loki, etcd watch, Grafana	
L5 Analysis	Turning measurements into a verdict and into evidence	RTO decomposer, threshold evaluator, Elle checker, report and evidence archive	

2.1 Execution flow

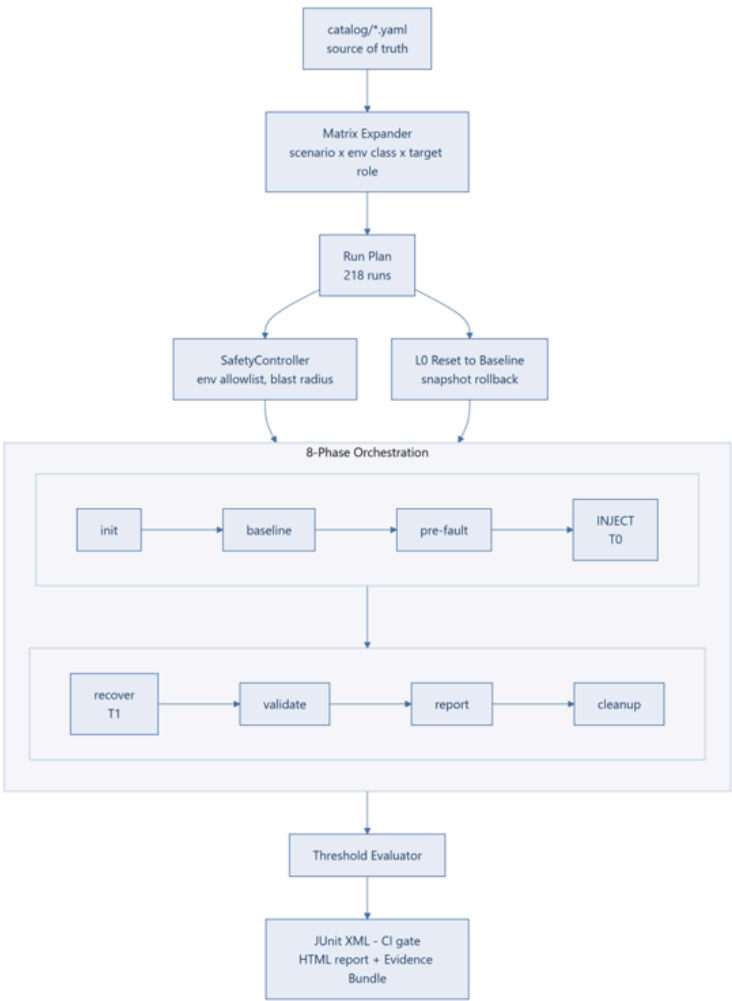


Figure 2 — From catalog to verdict. The run plan is derived, never hand-maintained.

3. L0 — Environment and Provisioning

The framework treats environment as a test axis, so provisioning is part of the harness rather than a prerequisite to it. L0 must produce any of E1–E4 on demand and reset it to a known baseline between runs.

Class	Provisioned by	Power-loss control path	Notes
E1 bare metal	Ansible on pre-racked nodes (MAAS/PXE optional)	ipmitool chassis power off / Redfish	Strongest NL-D evidence. Requires BMC access.

E2 dedicated-host VM	Terraform libvirt provider plus Ansible	virsh destroy	The only class where we can inject an <i>external</i> noisy neighbour — a co-located VM we own
E3 shared cloud VM	Terraform (AWS/Azure/GCP) plus Ansible	Cloud API force-stop	Noisy neighbour observed, never injected
E4 burstable	Terraform, burstable instance family	Cloud API force-stop	Credit depletion is self-inflictible

Two provisioning decisions that cannot be retrofitted

1. **PGDATA and pg_wal must sit on a device-mapper target from day one.** dm-flakey / dm-delay / dm-error (NL-D-03, NL-N-03, and the storage faults generally) require the volume to already be a DM device. You cannot insert a DM layer under a live data directory mid-test. Bake it into the Ansible storage role: LVM → dm-linear → optional dm-flakey shim → filesystem.
2. **Reset must be snapshot-based, not rebuild-based.** With 218 runs and many destructive scenarios, a full re-provision per run is fatal to the schedule. LVM or ZFS snapshot rollback resets a node in seconds; a rebuild takes minutes. This single decision is the difference between a full matrix completing in days versus weeks (§14).

3.1 Environment profile

Each class is described by a profile the harness consumes, so that scenarios stay environment-agnostic. A scenario names a *fault type*; the profile decides which driver serves it.

```
envs/e2-dedicated-vm.yaml
```

```
class: E2
```

```

storage_class: local_nvme      # local_nvme | bbu_raid | san |
network_block | nfs

power_control:

  driver: libvirt              # libvirt | ipmi | redfish | aws | azure | gcp
  uri: qemu+ssh://lab-host/system

noisy_neighbour:

  injectable: true             # false for E3 -- the neighbour is not ours
  driver: co_located_vm

storage_fault:

  dm_shim: true                # dm-flakey available in the storage path

reset:

  driver: lvm_snapshot
  snapshot: baseline-clean

sla_tier_target: platinum

```

4. L1 — Control Plane and the Catalog

4.1 The catalog is the source of truth

The framework requires one machine-readable catalog from which the documents are generated. That same file drives test parametrization, so the scenario definition and the test run are never allowed to drift.

```

catalog/NL/NL-D-02.yaml

id: NL-D-02
name: Durability under host power loss
tier: node_local
category: NL-D
applies_to: [standalone, distdb.pc, distdb.qc, distdb.worker]
environment:

  classes: [E1, E2, E3, E4]

```

```

env_sensitive: true          # → expands across all supported classes

steady_state: {tps_min: 800, p99_latency_ms_max: 50, duration_s: 120}

workload: {profile: oltp_write_heavy, concurrency: 64,
transaction_markers: true}

fault:

  type: host_power_loss

  driver: power_control      # resolved from the env profile, not hard-
coded

  target: primary

blast_radius: {max_nodes_affected: 1, environments: [lab, staging]}

abort_if: ["replication_lag_s > 300", "secondary_node_unhealthy"]

measure: [rpo_txn, rto_first_write_s, rto_to_slo_s, corruption_count,
amcheck_errors]

accept:                      # evaluated mechanically -- no human reading

  • rpo_txn == 0

  • rto_first_write_s <= 60

  • corruption_count == 0

priority: P0

```

Everything downstream is generated from this [file: the](#) framework catalog tables, the CSV tracker, pytest parametrization, and the framework's own validation checks. The schema is enforced with **Pydantic v2**, and the six mechanical checks run in CI on every catalog change — so a documentation defect becomes a build failure rather than a finding at the next review.

4.2 Orchestrator

The eight-phase machine is retained unchanged: init → baseline → pre_fault → fault_inject(T0) → recovery(T1) → validate → report → cleanup. Three additions:

- **Matrix expander** — reads env_sensitive and applies_to and produces the run plan: 21 environment-sensitive scenarios × 4 classes, plus 134 run once, = **218 runs**.

- **Reset phase** before init, delegating to the L0 reset driver.
- **State-triggered second injection** for the compound category (§4.3).

4.3 State-triggered injection (compound scenarios)

The nine compound scenarios in framework category **CL-X** inject a second fault *while the system is still recovering from the first*. The trigger is not a timer.

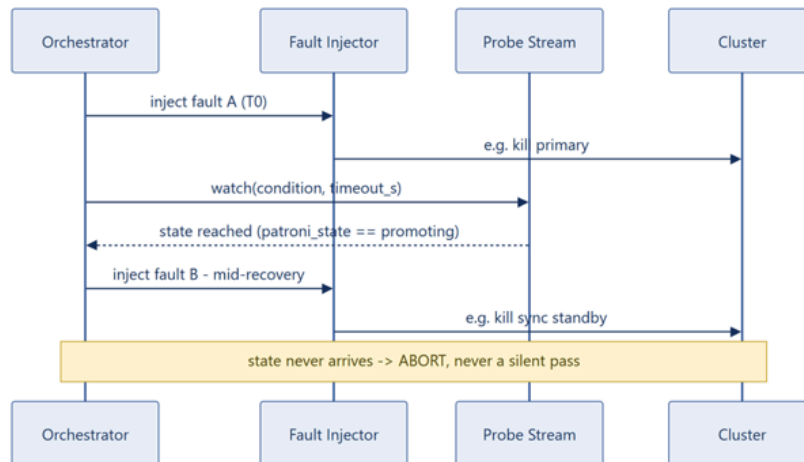


Figure 3 — Fault B is armed on a predicate over the probe stream, and the absence of the state is an abort.

Why a timer is the wrong mechanism

“Kill the sync standby 20 seconds after the primary dies” and “kill it the instant Patroni enters promoting” are different tests. Only the second reliably lands inside the fragile window, and the window's position moves from run to run with cluster timing.

A timer-based implementation produces a scenario that *looks* like it passed while frequently missing the condition it was written to exercise — the same class of silent false-negative as a mis-measuring RTO decomposer.

```
class CompoundOrchestrator(TestOrchestrator):
```

```
    """Arms fault B on a predicate over the live probe stream."""
```

```
    async def _phase_fault_inject(self, state, cfg, adapter):
```

```
        await self.injector.inject(cfg.fault)          # fault A, at T0
```

```

        trigger = cfg.compound.fault_b.trigger          # on_state | on_metric |
after_phase

        armed  = self.probes.watch(cfg.compound.fault_b.condition,
                                   timeout_s=cfg.compound.fault_b.timeout_s)

        fired = await armed

        if not fired:

            # The state never arrived. This is an ABORT, not a pass -- a
compound

            # scenario that silently degrades to a single-fault test is a false
negative.

            raise CompoundTriggerTimeout(cfg.compound.fault_b.condition)

        state.timing.t_fault_b = self.clock.now()

        await self.injector.inject(cfg.compound.fault_b) # fault B, mid-
recovery

        return state

```

The probe stream already emits every state transition these predicates need (patroni_state, rebuild_pct, backup_pct, leader-key events), so this is orchestration on top of existing measurement rather than new instrumentation.

Safety: the blast-radius ceiling rises to 2 for this category only

Every other scenario is capped at max_nodes_affected: 1. A compound scenario is by definition two concurrent faults, so CL-X carries a blast_radius_override of 2. The Safety Controller must treat this as a **category-scoped exception with its own hard ceiling** — not as a global relaxation, and not as an override any scenario can set for itself. Any scenario outside CL-X requesting more than one node is a configuration error and must fail closed.

The injection ledger matters more here too: two active injections across two hosts means a crashed harness can strand twice as much state. Both must be journalled before either fires.

5. L2 — Fault Injection

One FaultInjector interface, several drivers. The scenario names a *fault type*; the environment profile decides which driver serves it. This is what lets the same scenario run on bare metal and on a cloud VM without edit.

Fault type	Primary tool	Scenarios	Build / buy
Process kill / stop	systemctl kill -s SIGKILL, pg_ctl -m immediate over SSH	NL-C-, <i>CL-F-02</i> , <i>DX-M-</i>	chaosd, or trivial to build
CPU / memory / IO limit	systemctl set-property CPUQuota=, MemoryMax=, IOReadBandwidthMax=	NL-N-02, NL-R-*	build — thin systemd wrapper
Resource stress	stress-ng, fio	NL-R-06/07, NL-N-01	buy
Network delay / loss / rate	tc netem	CL-N-01...04	chaosd
Network partition	nftables / iptables rules	CL-S-, <i>DX-P-</i>	build — needs topology awareness
Proxy-level fault	toxiproxy	CL-C-*, CL-N-05	buy
Storage write loss / latency	dm-flakey, dm-delay, dm-error	NL-D-03, NL-N-03	build — DM table swap
Filesystem full	fallocate	NL-R-03, NL-W-07	build
Power loss	virsh destroy / ipmitool / Redfish / cloud force-stop	NL-D-02/05/06, <i>CL-F-03</i>	build — one per environment class

Clock	libfaketime, chronyc	NL-M-08	buy	
Data / WAL corruption	Targeted byte writes plus pg_waldump to locate structures	NL-I-*, NL-W-03	build — must be precise	
Counter pre- seeding	pg_resetwal -x / -m	NL-M-01/02, DX- T-10	build	

Evaluate chaosd before writing adapters

chaosd — the standalone, non-Kubernetes component of Chaos Mesh — covers process, network, disk, stress, JVM and clock faults on bare metal and VMs. It plausibly replaces four of the rows above behind a single client.

It does not cover power loss, device-mapper write loss, or targeted structural corruption — the three highest-value classes — so it is a complement, not a replacement. Recommendation: build the OS/SSH driver first (it is small and always available), then evaluate chaosd as a second driver behind the same interface.

6. L2 — Workload and Transaction Markers

6.1 Two workload engines

Engine	Used for	Why
pgbench	Baseline TPS and latency, steady-state soak, backup-under-load	Standard, well-understood numbers; custom scripts via -f

Custom asyncpg driver	Every RPO-measuring scenario; all DistDB multi-shard workloads	Transaction markers and 2PC-mode control need application-level logic that pgbench cannot express
Elle list-append profile (<i>same driver, distinct workload</i>)	The 18 history-checked scenarios (§6.3)	Elle infers dependencies from append order, which requires a purpose-built operation shape rather than generic OLTP

6.2 Transaction marker protocol — the RPO measurement of record

The marker journal must not live on the database host

If the marker journal sits on the node under test, a power-loss scenario destroys the evidence along with the database. **The workload driver and its marker journals run on a separate host** from the target. This is an architectural requirement, not an implementation detail.

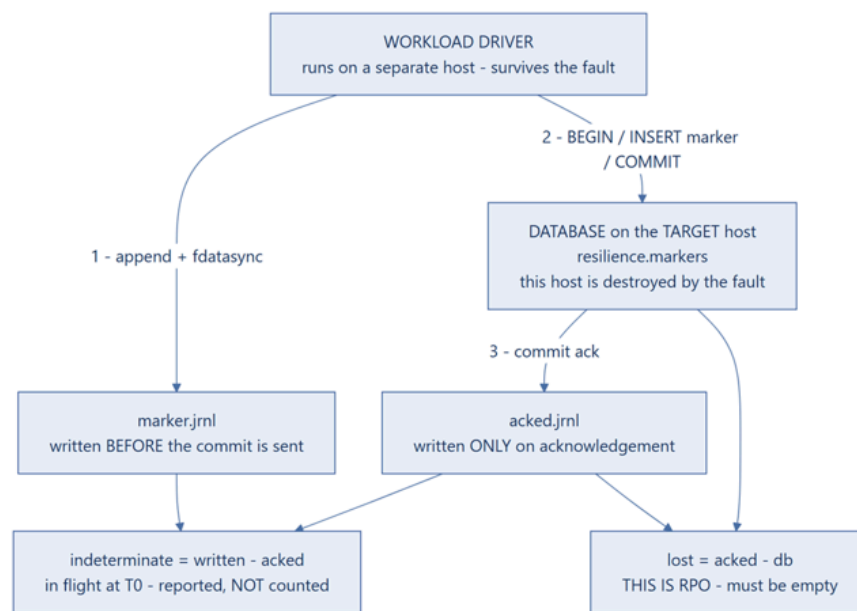


Figure 4 — The marker protocol. Two journals on the driver host, two set differences after recovery.

for each transaction:

```
seq += 1; uuid = uuid4()
```

```
append {seq, uuid, t_pre} to marker.jrnl; fdatsync(marker.jrnl) # driver  
host
```

```
BEGIN; INSERT INTO resilience.markers(uuid, seq, ts); COMMIT;
```

```
on COMMIT ack: append {uuid, t_ack} to acked.jrnl;  
fdatsync(acked.jrnl)
```

after recovery:

```
lost      = acked_set - db_set      # ← RPO. MUST be empty.
```

```
indeterminate = written_set - acked_set # in flight at T0; reported, NOT  
counted
```

Why the indeterminate set is reported separately

A transaction whose COMMIT was sent but never acknowledged has no defined outcome from the client's perspective — the database is permitted to have committed it or not. Counting those as RPO violations produces false failures; ignoring them silently hides a real class of ambiguity. They are measured, reported, and excluded from the RPO predicate.

6.3 Operation history export — feeding Elle

Eighteen scenarios additionally require transactional-anomaly checking. The workload driver therefore emits a **Jepsen-format operation history** alongside the marker journals. Elle checks that history offline; Jepsen is not adopted as a test framework (§17).

```
# Elle's list-append profile -- strongest inference, maps cleanly to SQL
```

```
CREATE TABLE resilience.lists (k int PRIMARY KEY, v text);
```

```
append k v → INSERT INTO lists VALUES (k,v)
```

```
ON CONFLICT (k) DO UPDATE SET v = lists.v || ';' || excluded.v
```

```
read k → SELECT v FROM lists WHERE k = k
```

```
# history record -- one line per event, harness clock

{:index 0 :type :invoke :f :txn :process 7 :time 1710_000_000 :value
[:append 3 12] [:r 5 nil]]}

{:index 1 :type :ok :f :txn :process 7 :time 1710_000_412 :value [:append
3 12] [:r 5 [1 4 9]]}]

{:index 2 :type :info :f :txn :process 9 :error :indeterminate} # ← NOT :fail
```

The one mistake that invalidates the whole check

A client may emit `:fail` **only when it is certain the transaction aborted** — an explicit serialization failure or constraint violation returned by the server. A network timeout, a connection reset, or a killed primary is `:info` (indeterminate), **never** `:fail`.

Elle treats `:fail` as “this definitely did not happen”, and will report bogus G1a aborted-read anomalies if an operation that actually committed is recorded as failed. Since these 18 scenarios inject faults precisely at commit time, almost every ambiguous outcome falls into this class.

Getting this wrong produces confident, plausible, wrong results — and it must be unit-tested against recorded fixtures.

DistDB key selection

For DX scenarios the checker is only meaningful if transactions actually span shards. Key generation is therefore shard-aware: the driver resolves shard placement from the coordinator metadata and deliberately composes transactions whose keys land on **different worker clusters**, with 2PC enabled and disabled as separate runs. A history in which every transaction happened to be single-shard proves nothing about distributed isolation.

7. Measurement Architecture

This is where resilience harnesses usually fail. The scenarios carry numeric thresholds such as $rto_first_write_s \leq 60$ and $p99 \leq 5 \times \text{baseline}$; something has to produce those numbers with enough resolution and clock discipline to be trustworthy.

7.1 Prometheus is context, not measurement

A default Prometheus scrape interval is 15 s. Failover budgets in this framework are 30–60 s. **Measuring a 30-second RTO with a 15-second sampler yields $\pm 50\%$ error** — the measurement cannot support the acceptance criterion. Tightening the scrape to 1 s helps but still cannot see sub-second transitions, and it degrades the monitoring stack under exactly the load conditions the tests create.

Decision: in-harness async probes at 100–250 ms are the measurement of record. Prometheus, node_exporter and postgres_exporter provide surrounding context and trend data, and are never the source of a pass/fail value.

7.2 RTO decomposition — signal sources

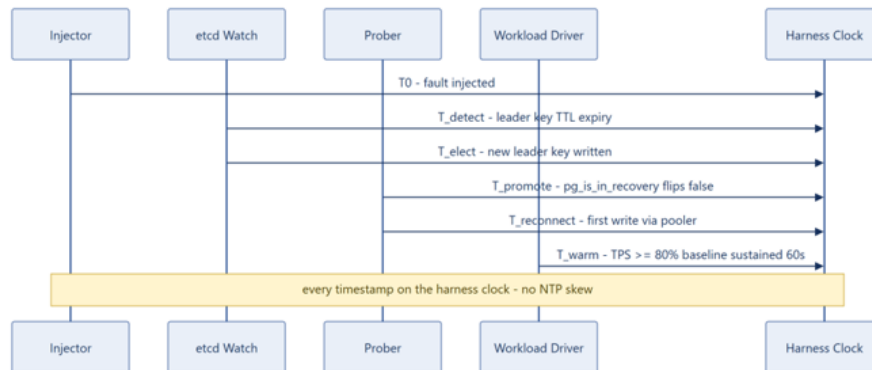


Figure 5 — Every RTO component is captured from a harness-observable signal.

Component	Signal	Captured by	Clock
T0	Injection call returns, or BMC confirms power off	Injector	Harness
T_detect	etcd leader key TTL expiry; Patroni “demoted self” / lock-loss log line	etcd Watch plus journald tail	Harness (watch event)
T_elect	New owner written to the	etcd Watch	Harness

	etcd leader key		
T_promote	pg_is_in_recover y() flips false; “ready to accept connections”	Prober plus log tail	Harness (prober)
T_reconnect	First successful write <i>through the pooler</i>	Prober	Harness
T_warm	TPS $\geq$ 80% baseline and p99 $\leq 1.5\times$ baseline, sustained 60 s	Workload driver	Harness

Clock discipline

Every timestamp above is taken on the **harness clock** wherever possible — the etcd watch event, the probe result and the workload sample are all observed locally, so no cross-host clock comparison is involved.

The exception is PostgreSQL and Patroni log timestamps, which originate on the node. The harness therefore measures each node's NTP offset at test start, corrects log-derived timestamps, and **prefers a harness-observable signal whenever one exists for the same event**. Without this, NTP skew of a few hundred milliseconds silently corrupts the RTO decomposition — which is precisely the number the whole framework is tuned against.

7.3 Probe design

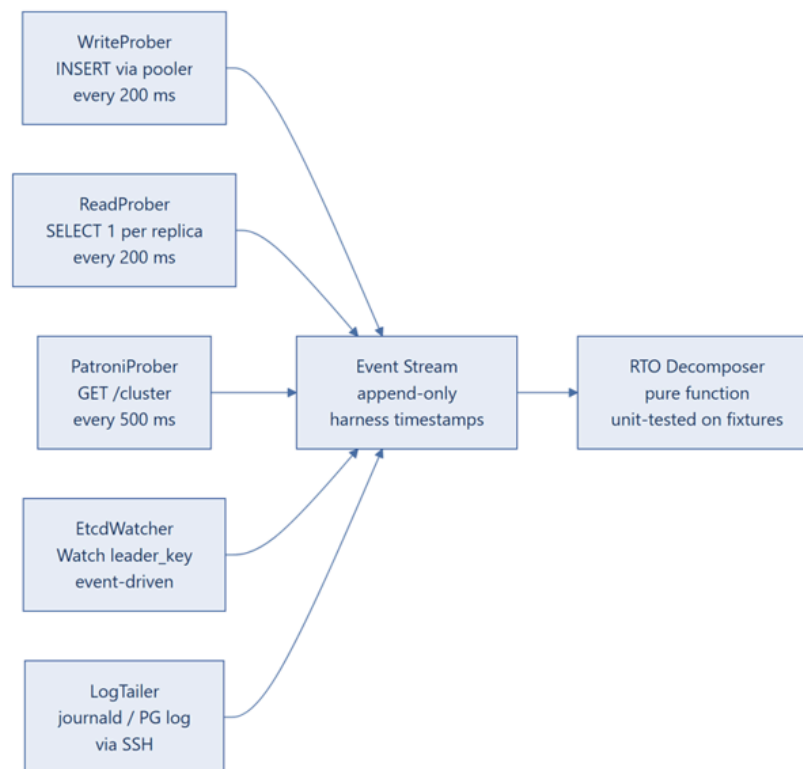


Figure 6 — Probers feed one append-only event stream; the decomposer is a pure function over it.

Probers write to an append-only event stream with harness timestamps. The RTO decomposer is a pure function over that stream, which means it is unit-testable against recorded fixtures — important, because a decomposer bug silently produces plausible-looking wrong numbers rather than crashing.

8. L3 — Target Adapters and Role Resolution

Role resolution is the single capability that makes one catalog serve both products. A Tier-1 scenario is written once against the adapter interface and executed against every resolved target.

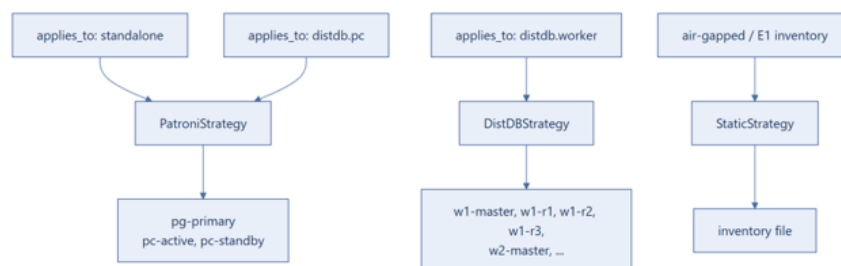


Figure 7 — `applies_to` is expanded to concrete hosts by a strategy per topology.

Adapter surface	Responsibility
PostgreSQLAdapter	Connections, try_write(), cluster info, replication status, integrity checks, restart-as-replica
DistDBAdapter	Extends the above with shard placement, 2PC state (the pg_prepared_xacts equivalent), GXID inspection, metadata version, reference-table consistency
PatroniClient	REST /cluster, /switchover, /failover; watchdog state

Both adapters implement one BaseDatabaseAdapter, so the scenario body never learns which product it is running against.

9. L4 — Observability

Component	Tool	Role in the harness
Metrics	Prometheus + node_exporter + postgres_exporter	Context and trend only. 1 s scrape during test windows; never a pass/fail source
Logs	Loki + promtail, plus a direct journald tail during the fault window	Patroni and PostgreSQL event corroboration; root-cause evidence
Dashboards	Grafana	Human review; per-run annotated timeline
Consensus events	etcd Watch API (direct)	Authoritative T_detect and T_elect
Trend store	TimescaleDB, or Prometheus long-term storage	Regression detection across runs and releases

10. L5 — Validation, Analysis and Evidence

10.1 Integrity tooling — the correct tool per corruption class

Tool	Detects	Scenarios
pg_amcheck / amcheck	Heap and btree structural corruption; -- heapallindexed cross- check	NL-I-01/02/07
`pg_visibility` — pg_check_visible(), pg_check_frozen()	Visibility-map inconsistency. amcheck does not verify the VM	NL-I-05
pg_checksums	Offline page checksum verification	NL-D-05, NL-I-01
pg_controldata, pg_waldump	Control-file state; WAL record inspection and truncation LSN	NL-I-04, NL-W-03
pgbackrest verify	Backup repository and chain integrity	CL-B-06/07
Invariant checker (custom)	FK integrity, row counts, ordered-range checksums	NL-I-08, CL-R-07, DX- P-05

Replica comparison must be range-checksummed, not dumped

CL-R-07 samples continuously and DX-P-05 must converge on large datasets; a full table comparison will not finish. Use `md5(string_agg(...))` or a hash aggregate over primary-key ranges, executed in parallel per range, comparing only range digests.

10.2 Threshold evaluator

The accept: predicates in the catalog are evaluated as expressions against the measured-value dictionary. No natural-language interpretation, no human judgement:

```

measured = {"rpo_txn": 0, "rto_first_write_s": 24.3, "corruption_count": 0,
...}

verdict = all(safe_eval(pred, measured) for pred in scenario.accept)

```

Failures record the predicate, the measured value and the margin — so a near-miss at 58 s against a 60 s budget is visible as a trend before it becomes a regression.

10.3 Consistency checking with Elle

Elle is the transactional-anomaly checker from the Jepsen project. It consumes a recorded history and reports isolation anomalies with a minimal counterexample cycle. We adopt **the checker, not the framework** (§17).

Anomaly	Meaning	Why it matters here
G0	Write cycle	Concurrent writes ordered inconsistently across shards
G1a	Aborted read	A reader saw a value from a transaction that rolled back — the classic partial-commit signature when 2PC is disabled
G1b	Intermediate read	A reader saw a non-final value mid-transaction
G1c	Cyclic information flow	Transactions mutually observed each other — impossible under serializability
G-single	Read skew	A read spanning two shards saw them at different logical times
G2	Anti-dependency cycle	Write skew; the anomaly snapshot

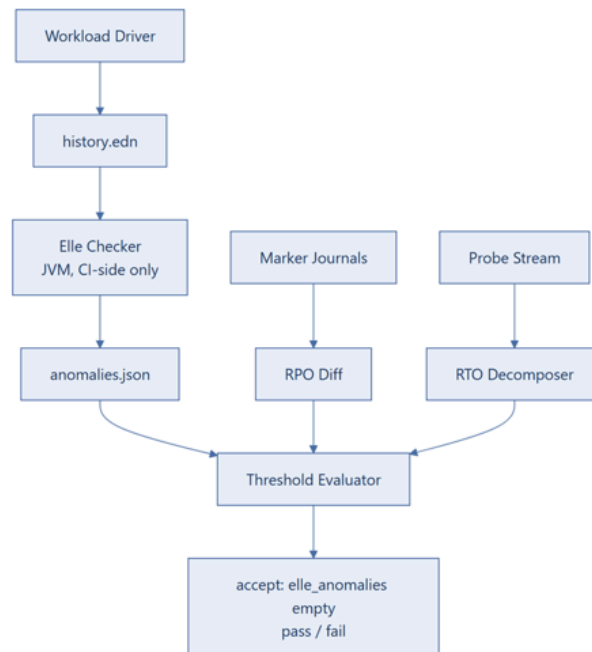


Figure 8 — Elle's verdict is one more measured value the threshold evaluator consumes.

Because the verdict enters the same measured-value dictionary as timing and integrity results (§10.2), a consistency failure gates the build exactly like an RTO breach.

Honest note on the dependency

Elle is Clojure and runs on the JVM. Adopting the checker alone keeps Clojure out of the harness, the product and the EQS binary — but it does add a **JVM to the CI pipeline** as a build-time checker dependency. That is a real cost, and a bounded one: the checker is invoked as `history.edn → verdict.json` and nothing else in the stack touches it.

`elle-cli` wraps Elle and Knossos for exactly this workflow. **Verify its current maintenance status before committing** (§18, decision 8); the fallback is a ~50-line Clojure shim calling `elle.list-append/check` directly, vendored and pinned.

Practical bounds

Cycle detection is superlinear in history size, so histories are kept in the 10k–100k operation range per run rather than recording an entire soak. For

the 18 checked scenarios this is ample — the interesting window is the few thousand operations either side of T0.

10.4 Outputs

Artefact	Format	Consumer
CI gate	JUnit XML	Jenkins — fails the build on any P0 failure
Run report	HTML (Jinja2) with an annotated timeline	Engineering
Machine result	JSON	Trend store, regression analysis
Evidence bundle	tar.zst → MinIO/S3	Audit: probe stream, logs, metric snapshot, amcheck output, marker journals
Qualification report	Signed PDF/HTML	Customer (EQS, §11)

11. Environment Qualification Suite Packaging

EQS runs on **customer infrastructure**, so its constraints differ sharply from the main harness: minimal footprint, no assumed runtime, safe defaults, and a defensible report.

Suite	Content	Runs against	Destructive?
EQS-C	Configuration assertions: checksums, fsync, synchronous_commit, NTP, consensus placement	Live or pre-production	No — read-only
EQS-P	Steal %, cgroup throttling, fsync	Live (low impact)	No — observes, never injects

	p99, 24 h variance — measurement only		
EQS-D	NL-D- 01/02/03/05/06 — power loss, write-loss injection	Scratch instance only	Yes — destroys the instance

EQS-D safety gate

EQS-D deliberately destroys the database it runs against. It must refuse to start unless the target is explicitly marked disposable — a required --target-is-disposable flag **plus** a sentinel table or file the operator creates by hand. A qualification tool that can be pointed at production by accident is a liability, not an asset.

11.1 Packaging recommendation

Option	Trade-off	Verdict
PyInstaller single binary	Reuses harness code directly; ~30–50 MB; per-platform builds	Phase 2 choice — fastest path, no code duplication
Go static binary	~10 MB, zero runtime dependencies, trivially cross-compiled; requires reimplementing the durability probes	Migrate if customer footprint or Python availability becomes an objection
Container image	Clean isolation	Rejected — the product is not containerized; requiring a runtime the customer does not otherwise run

is a poor first
impression

12. Technology Stack — Decisions and Rejected Alternatives

Each row records what was chosen, why, and what was considered and rejected. The rejected column is deliberately preserved: it is the part that stops a decision being relitigated every quarter.

Concern	Choice	Rationale	Rejected
Language	Python 3.11+, asyncio	Existing reference implementation; readable by the DBAs who will extend it; strong PostgreSQL ecosystem	Go (better binary story — retained for EQS only); Rust (team velocity)
DB driver	asyncpg	Highest throughput for marker-driven load generation; native protocol	psycopg3 — fine, but the reference code is asyncpg
Test runner	pytest + pytest- xdist	JUnit XML, parametrization, fixtures and CI integration for free	A bespoke runner — reinvents reporting for no gain
Catalog schema	YAML + Pydantic v2	Human-editable, machine- validated, generates both documents and tests	JSON (poor ergonomics); CSV (no nesting)

Remote execution	asynssh	Async, key-based, precise timing control	Ansible ad-hoc — startup latency ruins injection timing
Config management	Ansible	Agentless, ideal for non-containerized fleets	Puppet/Chef (agent overhead); Helm (not applicable)
Cloud provisioning	Terraform	E3/E4 reproducibility across providers	Hand-built instances
Lab virtualization	libvirt / KVM	virsh destroy gives true force-off semantics	VirtualBox — weaker power-off fidelity
State reset	LVM or ZFS snapshots	Seconds per reset; this is what makes 218 runs schedulable	Full re-provision — schedule-fatal
Fault injection	OS/SSH driver; chaosd as a second driver; toxiproxy for proxy faults	Works on E1–E4 with no orchestrator	Chaos Mesh CRDs, LitmusChaos — Kubernetes-only
Workload	pgbench + custom asyncpq driver	Standard baselines plus marker and 2PC control	sysbench (less PostgreSQL-native); HammerDB (heavy)
Metrics	Prometheus + exporters	Context and trend	Prometheus as a measurement source — too coarse (§7.1)

Logs	Loki + promtail	Correlates cluster events to the run timeline	ELK — heavier for the same need
HA / backup under test	Patroni + etcd; pgBackRest; PgBouncer	These are the product's own stack	repmgr (weaker fencing); pg_basebackup alone (no verify, no incremental)
Consistency checking	Elle (via elle-cli or a pinned Clojure shim), fed by an exported history	The only practical way to detect isolation anomalies — G1a/G1c/G-single/G2 — that invariant checks structurally cannot see	Jepsen as the test framework — covers only ~14% of scenarios and cannot inject power loss, dm-flakey or measure RTO (§17); hand-rolled anomaly detection — this is a solved research problem
Evidence store	MinIO (S3 API)	Self-hosted, audit retention	Filesystem only — no retention policy
Reporting	Jinja2 → HTML; JUnit XML	Matches the existing deliverable style	Allure — extra infrastructure
CI	Jenkins, dedicated agents	Long jobs, hardware and BMC pinning, existing pipeline design	GitHub Actions — no bare-metal or IPMI access

13. Repository Structure:

```
resilience/
|-- catalog/          # SOURCE OF TRUTH -- one YAML per scenario
|  |-- NL/ CL/ DX/
|  \-- schema.py      # Pydantic v2 models + the 6 validation checks
|-- envs/             # E1-E4 environment profiles
|-- resilience_tests/
|  |-- control/       # orchestrator, phase machine, matrix expander,
safety
|  |-- execution/
|  |  |-- injectors/  # os_ssh, systemd, netem, nftables, dm, power,
|  |  |              # chaosd, toxiproxy
|  |  |-- workload/   # pgbench wrapper, asyncpg driver, markers,
|  |  |              # elle_profile.py (list-append), history_writer.py
|  |  \-- probes/     # write/read/patroni probers, etcd watcher, log
tailer
|  |-- adapters/      # base, postgresql/, distdb/, target_resolver.py
|  |-- observability/ # prometheus, loki, event stream
|  \-- analysis/      # rto_decomposer, threshold_eval, report,
evidence,
|                    # elle_checker.py (subprocess → JVM checker)
|-- vendor/elle/      # pinned Elle checker + shim; JVM used only here
|-- eqs/              # Environment Qualification Suite (packaged
separately)
|-- infra/
|  |-- terraform/     # E2 libvirt, E3/E4 cloud
|  \-- ansible/       # postgres, patroni, etcd, pgbouncer, pgbackrest,
|                    # storage (LVM + dm shim), monitoring, lab-neighbour
|-- tests/            # unit tests for the harness itself
```

```
|
# -- incl. decomposer fixtures
|-- reports/ evidence/
\-- ci/
# Jenkinsfile, pipeline library
```

tests/ is not optional

The RTO decomposer, the threshold evaluator and the marker differ are pure functions whose bugs produce plausible-looking wrong numbers rather than crashes. They are unit-tested against recorded probe-stream fixtures. **A harness that silently mis-measures is worse than no harness** — it manufactures false confidence and there is nothing in the output to suggest anything is wrong.

14. Execution Model, Cadence and Capacity

14.1 Capacity arithmetic

218 runs x (~25 min execute + ~5 min snapshot reset) ~= 109 hours
serial ~= 4.5 days

... plus long-running outliers: NL-M-04 (4 h), NL-N-08 (4 h), CL-B-05 (volume-bound)

With 4 independent environment stacks running in parallel: ~= 27 h ~= 1.1 days wall clock

Full-matrix execution therefore cannot be nightly. The cadence follows from the arithmetic rather than from preference:

Cadence	Scope	Runs	Wall clock	Gate
Per commit	Harness unit tests plus catalog validation checks	—	< 5 min	Blocks merge

Nightly	P0 fast subset, reference environment (E2), standalone and DistDB	~35	~4 h	Blocks the release candidate
Weekly	All P0 and P1, reference environment	151	~18 h	Release gate
Per release	Full matrix — all scenarios × all environment classes	218	~1.2 days	Ships the supported-environment matrix
Per deployment	EQS-C/P/D on customer infrastructure	~12	~24 h (EQS-P variance window)	Assigns the SLA tier

Four parallel stacks is a hardware requirement, not a nice-to-have

It is what makes a per-release full matrix feasible. Fewer stacks pushes the full matrix past a working week, and it will then quietly stop being run — the standard failure mode for resilience programmes.

15. Safety Architecture

Control	Behaviour
Environment allowlist	The harness refuses to run unless the target fingerprint (hostname pattern, sentinel table, inventory tag) is on the allowlist. Default deny.

Destructive-scenario gate	NL-D and EQS-D additionally require an explicit disposable-target flag plus an operator-created sentinel
Blast radius cap	max_nodes_affected enforced before injection; exceeded → abort. Hard ceiling 1 for every category except CL-X compound , whose ceiling is 2 . A scenario outside CL-X requesting more than one node is a configuration error and fails closed — the override is category-scoped, not scenario-settable (§4.3).
Abort conditions	abort_if predicates evaluated continuously against the probe stream
Kill switch	A single command reverses all active injections across all hosts and restores DM tables, nftables rules and systemd properties
Injection ledger	Every injection is journalled before execution, so a crashed harness can still be cleaned up — the harness must not be able to strand a node in a faulted state
Timeouts	Every phase is bounded; an unbounded wait is a defect

16. Build Sequence

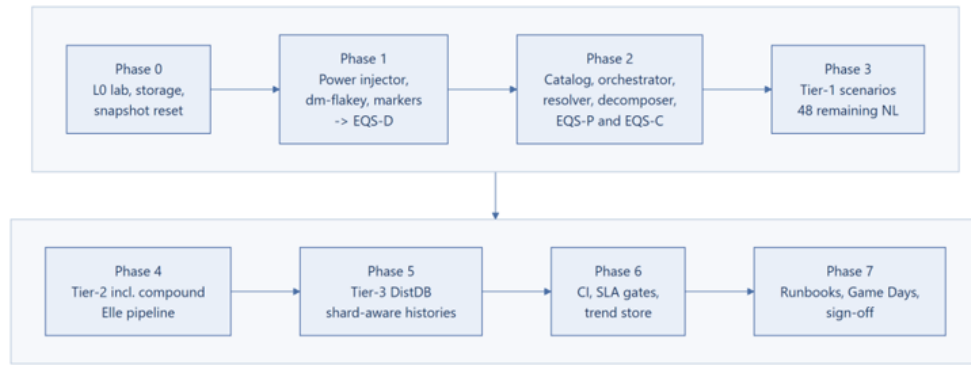


Figure 9 — Build order, mapped onto the framework's roadmap phases.

Phase	Deliverable	Proves
0	L0: Ansible roles (including LVM plus the dm shim), Terraform for E2/E3, snapshot reset, one stack per class	Any environment class is reproducible and resets in seconds
1	Power-loss injector, dm-flakey driver, marker system on a separate host, NL-D-01...06 → EQS-D	Durability is evidenced per storage class. The RPO = 0 claim becomes defensible
2	Catalog, schema and generators; orchestrator; TargetResolver; probe and event stream; RTO decomposer; threshold evaluator; EQS-P and EQS-C packaged	One Tier-1 scenario runs end to end against every node role; EQS runs on arbitrary customer infrastructure
3	The remaining 48 NL scenarios; corruption injectors; pg_resetwal pre-seeding	All Tier-1 P0 pass on both products

4	55 CL scenarios including the 9 compound CL-X; nftables partition driver; toxiproxy; etcd watcher hardening. Elle pipeline: list-append profile, history writer, pinned checker, fixture-based unit tests for :fail-versus-:info classification	All Tier-2 P0 pass; the RTO budget is decomposed; the first 6 history-checked scenarios are clean
5	46 DX scenarios; DistDB adapter; multi-shard workload; 2PC state inspection; shard-aware key generation so that histories genuinely span worker clusters	All Tier-3 P0 pass; the remaining 12 history-checked scenarios are clean with 2PC both enabled and disabled
6	Jenkins pipeline, SLA gates, trend store, Grafana dashboards	Regressions are caught automatically
7	Runbooks, Game Days, audit evidence packaging, training	Sign-off

Phase 1 deliberately precedes the framework build. It requires only the power injector, the marker system and a results file — not the catalog, the orchestrator or the reporting stack. Building it first closes the durability gap in weeks rather than months, and produces EQS-D, which is the artefact with the most immediate commercial value.

17. Settled Technical Decisions

Recorded because each one is the sort of question that otherwise resurfaces every quarter, and because the reasoning matters more than the answer.

17.1 Jepsen: adopt the checker, not the framework

Decision

Elle is adopted as a checker over an exported history. Jepsen is not adopted as a test framework.

Consideration	Finding
Coverage	Jepsen's natural territory is 21 of 155 scenarios — roughly 14%.
Fault range	Jepsen's nemesis cannot inject power loss, device-mapper write loss, cgroup throttling or CPU steal.
What it measures	Correctness rather than time — leaving the timing criteria and the storage/integrity criteria unserved.
Team and stack cost	Adopting it wholesale would put Clojure on the critical path for a Python team, and would force a separate EQS build in any case.
Cost of the chosen path	Exporting a history and checking it with Elle captures the unique value — isolation-anomaly detection — in roughly 2–3 weeks rather than 6–10, and keeps Clojure out of everything except a CI-side checker.

18. Open Technical Decisions

Each requires an owner and a decision before the phase named in the final column begins.

#	Decision	Why it matters	Needed by
1	How many parallel environment	Directly sets full-matrix wall clock (§14). Below four,	Phase 0

	stacks are funded?	per-release full-matrix execution stops being practical.	
2	Is BMC/IPMI access available on E1 lab nodes?	Without it, NL-D-02 and NL-D-06 fall back to hypervisor force-off on E2 — weaker evidence that must be disclosed in any compliance claim.	Phase 0
3	LVM or ZFS for snapshot reset?	ZFS gives cheaper snapshots and built-in checksums (useful corroboration for NL-I); LVM is more conventional and closer to typical customer storage. Choose one and standardise.	Phase 0
4	Adopt `chaosd`, or build the OS driver only?	chaosd removes perhaps four injector classes of work, but adds an agent to deploy and	Phase 2

		version on every target host.	
5	Which isolation level does each product claim, per configuration?	Elle checks a history <i>against a declared model</i>. PostgreSQL defaults to Read Committed; serializability requires SERIALIZABLE. For DistDB the claim must be stated separately for 2PC-enabled and 2PC-disabled DML. Without a documented claim there is nothing to falsify.	Phase 4
6	Is `elle-cli` currently maintained?	Determines whether the checker is an off-the-shelf binary or a small pinned Clojure shim we vendor. Low risk either way, but it should be settled before Phase 4 rather than discovered during it.	Phase 4

7	Does DistDB expose a GXID pre-seeding facility?	DX-T-10 is unexecutable without it. May require a product change — long lead time.	Phase 5
8	Does DistDB expose prepared-transaction and metadata-version state via SQL?	Determines whether the DistDB adapter can observe 2PC state directly or must infer it from logs. Affects roughly 11 DX-T scenarios.	Phase 5
9	What is the EQS distribution channel and support model?	Customer-run means versioning, documentation, support burden and a defensible report format. Field-engineer-only is materially cheaper.	Phase 2

19. Document Approval

Role	Name	Signature	Date
Chief Architect			
Product Owner			
DistDB Engineering Lead			

QA Lead			
Engineering Director			

Resilience Test Harness — Implementation Architecture

Version 1.0 · Database Reliability Engineering · CONFIDENTIAL — INTERNAL USE ONLY

Implements the Unified Resilience Testing Framework v1.0 · 155 scenarios · 218 runs

Review at each phase gate, or on any change to the framework catalog.